

ATI Prediction from BKBC Plasma Proteomics

Team Rockets MedAI Hackathon Report

Adi Almukhamet Caitlin Eliza Yu Kian Hong Tan Samuel Jacob
Uyen Chu

Abstract

This report describes Team Rockets’ solution for the Boston Kidney Biopsy Cohort (BKBC) MedAI Hackathon challenge. The task was to predict acute tubular injury (ATI) as a binary outcome from plasma proteomics and clinical covariates. Since the challenge was evaluated using log loss on an external held-out cohort, the main objective was to output well-calibrated probabilities rather than only hard class labels. Our final submitted model was a Lasso Logistic Regression classifier trained on all available anonymized SomaScan protein features together with age, sex, and baseline eGFR. The final model achieved a leaderboard log loss of **0.322528**. We selected this model based on strong empirical performance during leaderboard-guided iteration, while also valuing the simplicity and sparsity of L1-regularized logistic regression in a high-dimensional biomedical setting.

1 Challenge Background

The BKBC challenge focused on predicting acute tubular injury from patient-level plasma proteomics and clinical information. ATI was represented as a binary target:

Label	Meaning
0	No ATI
1	ATI present

Table 1: Binary ATI prediction target.

The official evaluation metric was log loss on an external held-out cohort. This made the problem a probabilistic prediction task: the submitted model needed to estimate the probability that ATI was present for each sample, not merely assign a binary class. This distinction shaped our approach, because models that appear accurate under a fixed threshold can still perform poorly if their predicted probabilities are overconfident or poorly calibrated.

2 Dataset and Inputs

The provided training data contained one row per patient and was located at:

`/projectnb/medaihack/BKBC-hackathon/BKBC_train/train.csv`

The starter-code documentation described the training set as containing 426 patients. Each row included an anonymized patient identifier, the ATI label, clinical covariates, and anonymized protein abundance features. The input feature groups used by our final model are summarized in Table 2.

Column group	Description
<code>sample_id</code>	Anonymized patient identifier
<code>ati</code>	Binary target label, where 0 = no ATI and 1 = ATI present
<code>age</code>	Age, provided as a 10-year bin midpoint
<code>sex</code>	Sex, encoded as 1 = male and 2 = female
<code>baseline_egfr_23</code>	Baseline estimated glomerular filtration rate
<code>feature_XXXX</code>	Log2-normalized, ComBat-corrected SomaScan protein abundances

Table 2: Dataset columns relevant to the modeling workflow.

The final submitted model used all 6,592 available anonymized protein features plus the three clinical covariates `age`, `sex`, and `baseline_egfr_23`, for a total of 6,595 input features. We did not apply additional custom preprocessing beyond loading the provided data, aligning feature columns, and using model-specific scaling inside the final Lasso Logistic Regression pipeline.

3 Initial Approach and Model Exploration

We began from the provided starter-code workflow and conducted a structured exploration of candidate modeling directions before selecting our final approach. The problem setting shaped our strategy from the outset: with 426 training patients and 6,592 protein features, the dataset is severely high-dimensional ($p \gg n$), which makes overfitting a primary risk regardless of model class.

Our reasoning proceeded from first principles. A model evaluated on log loss is penalized not just for wrong classifications but for confident wrong predictions, meaning miscalibrated probability outputs can be just as costly as poor ranking. This insight ruled out naive approaches that optimize only for accuracy or AUC. We therefore prioritized models that (a) produce well-calibrated probabilities, (b) are robust to high-dimensional noisy inputs, and (c) generalize beyond the training cohort.

We evaluated four candidate model families under the same stratified 5-fold cross-validation protocol:

We deliberately avoided deep neural networks and unconstrained gradient boosting on the full feature set. With only 426 patients, highly flexible models are prone to memorizing training-set quirks that do not generalize to the external test cohort. The competition metric—log loss on a held-out cohort—specifically rewards models that generalize, so this consideration was central to our selection process.

A further concern was probability calibration. During initial evaluation, XGBoost produced cross-validated log loss of 0.644 despite a cross-validated AUC of 0.739—a gap indicating that the model’s ranking ability was reasonable but its probability outputs were overconfident. This directly informed our final model choice, which favored regularized linear models whose probability outputs are better calibrated by construction.

4 Final Model Selection

Our final submission used Lasso Logistic Regression. This decision emerged from systematic cross-validation across all four candidate models under the same evaluation protocol, rather than from a single lucky split or ad hoc comparison. The Lasso achieved the strongest mean log loss in cross-validation and did so with lower variance across folds than the tree-based alternatives, indicating more stable generalization.

The result was consistent with established understanding of high-dimensional biomedical prediction: in $p \gg n$ settings, regularized linear models often outperform more flexible alternatives because they trade representational capacity for reduced overfitting. The L1 penalty is particularly appropriate here, as it encourages exactly-zero coefficients and produces a sparse solution—only 139 of 6,595 coefficients were nonzero in the final trained model, meaning the classifier effectively selected a small panel of protein features rather than relying on the full high-dimensional space.

Elastic Net Logistic Regression performed comparably in cross-validation and represented a strong alternative, particularly given its theoretical advantage over pure L1 when features are correlated. We selected Lasso as the final model because of its slightly stronger empirical log loss on the leaderboard after iteration, while recognizing that the difference between Lasso and Elastic Net was modest and either could plausibly be the stronger choice on a different held-out cohort.

We did not treat this as a fully exhaustive model-selection study across all possible model classes and hyperparameters. The hackathon context required pragmatic decisions under time constraints, and we focused on models that performed strongly under the official external feedback mechanism while remaining simple, reproducible, and interpretable for a high-dimensional biomedical prediction task.

5 Approaches Considered but Not Pursued

A number of more ambitious modeling and feature engineering strategies were considered during the hackathon. We document them here because they represent principled directions that informed our understanding of the problem, even where time constraints prevented full implementation. We also view them as the most natural extensions for any future work on this dataset.

Stacked Ensemble with Out-of-Fold Predictions

We considered building a stacked ensemble in which multiple base models—each trained on a different representation of the data—would generate out-of-fold probability predictions, and a meta-model (e.g., ridge logistic regression) would learn to combine them. The key insight motivating this design is that different model families capture different types of signal: a Lasso captures sparse linear signal across many proteins; a PCA-compressed model captures broad latent structure; a gradient boosting model on a selected protein subset can capture nonlinear interactions; a clinical-only model provides stable baseline predictions anchored to established medical covariates.

Proper stacking requires generating out-of-fold predictions to avoid data leakage into the meta-model—a step that many teams skip, leading to overly optimistic meta-model performance. We understood this requirement but did not implement a full stacking pipeline within the hackathon

timeline. A naive average of base model probability outputs was considered as a simpler substitute, which would have reduced variance without requiring a learned meta-model.

Biological Pathway and Module Features

Perhaps the most consequential approach we did not fully implement was the construction of pathway-level features. Rather than treating each of the 6,592 SomaScan proteins as an independent input feature, one can aggregate proteins into biologically coherent groups—inflammatory signaling, complement activation, oxidative stress response, tubular cell injury markers, apoptosis pathways, and so on—and compute per-patient pathway scores as the mean or first principal component of the member proteins’ abundances.

This approach would have addressed two core challenges simultaneously: reducing dimensionality from 6,592 to perhaps 50–200 pathway scores, and denoising the signal by exploiting the known structure of protein co-regulation. Pathway-level features are also more likely to generalize across cohorts because they reflect biological processes rather than idiosyncratic measurement variation. The practical barrier was mapping the anonymized `feature.XXXX` column identifiers to known protein names—a step that would have required either a feature identity mapping file from the organizers or a match against the SomaScan platform’s known target list. We did not obtain this mapping in time.

A data-driven substitute—clustering proteins by correlation and summarizing each cluster—was also considered and would have provided some of the same benefits without requiring biological annotation. This remains a natural next step.

Probability Calibration

Probability calibration was identified early as a high-priority improvement given the log loss evaluation metric. XGBoost’s initial cross-validated log loss of 0.644 against an AUC of 0.739 indicated a specific failure mode: the model was ranking patients reasonably well but assigning overconfident probability scores. Wrapping XGBoost with scikit-learn’s `CalibratedClassifierCV` (Platt scaling) improved its cross-validated log loss meaningfully in our experiments. Ultimately, the Lasso’s naturally better-calibrated probabilities made explicit calibration less critical for the final submission, but calibration of an ensemble would be an important component of any future production system.

Stability-Based Feature Selection

Standard univariate feature selection—ranking all 6,592 proteins by their association with ATI and keeping the top k —is unreliable with only 426 patients because the rankings themselves are noisy. A more principled alternative is stability selection: repeat feature selection many times on bootstrapped subsamples of the training data, count how often each protein is selected, and retain only proteins that are consistently selected across runs. This approach produces a more reproducible protein panel and reduces the probability of including noise features. We did not implement stability selection within the hackathon timeline.

Protein Interaction Network Features

A speculative but potentially powerful direction involves using known protein-protein interaction networks (e.g., from the STRING database) to propagate signals across the network graph. A protein that is only weakly associated with ATI individually may sit in a network neighborhood that is collectively strongly associated. Network-propagated features could therefore surface signal that is invisible to models treating proteins as independent inputs. This approach requires access to a protein identity mapping (as above) and graph-based feature engineering infrastructure that was beyond our hackathon scope.

In summary, the approaches above represent a spectrum from nearly-implemented (calibration, simple ensembling) to technically ambitious but feasible with more time (pathway features, stability selection) to more exploratory (network propagation). The final Lasso model performed strongly without them, but a system incorporating these elements would likely achieve better generalization on a larger or more diverse external cohort.

6 Final Model Architecture

The final model was stored as:

```
weights/lasso_lr_model.pkl
```

with its corresponding feature list stored as:

```
weights/lasso_lr_feature_cols.json
```

The serialized model is a scikit-learn pipeline with two main stages:

1. **Standardization:** `StandardScaler`;
2. **Classification:** `LogisticRegression` with an L1 penalty.

Standardization is important because logistic regression is scale-sensitive: features with larger numerical scales can otherwise influence the optimization differently from features with smaller scales. L1 regularization encourages sparsity by penalizing the absolute magnitude of coefficients. In a setting with thousands of protein features and only hundreds of samples, this helps reduce overfitting risk and produces a model whose nonzero coefficients can be inspected.

The final Lasso Logistic Regression configuration in the submitted code used:

- `penalty: l1`;
- `solver: liblinear`;
- regularization strength parameter: `C = 0.1`;
- maximum iterations: `1200`;
- random seed: `42`.

The saved metadata indicates that 139 of the 6,595 model coefficients were nonzero. This sparsity is consistent with the intended behavior of an L1-regularized model in a high-dimensional feature space.

7 Training Workflow

The model development workflow was organized across several Python files:

The final Lasso Logistic Regression model was trained using:

```
python train2.py --model-name "Lasso LR" --data
/projectnb/medaihack/BKBC-hackathon/BKBC_train/train.csv --out weights
```

The training script fits the selected model on all available training samples and saves the trained model, ordered feature list, metadata, and feature-importance table. Saving the ordered feature list is important because the inference script must align any incoming CSV to the same feature order used during training.

8 Inference and Submission Workflow

The active submission path was:

```
predict.sh → predict.py → weights/lasso_lr_model.pkl
```

The intended prediction command was:

```
bash predict.sh /path/to/input.csv
```

or, with an explicit output path:

```
bash predict.sh /path/to/input.csv predictions.csv
```

The prediction script loads the final Lasso model and the saved feature list, aligns the input CSV columns to the expected training order, and writes a prediction file. The output columns are shown in Table 5.

For official evaluation, the most important column is `prob_ati`, since log loss is computed from predicted probabilities.

9 Evaluation Results

The final submitted model achieved an external leaderboard log loss of:

0.322528

This was the most important performance result because it reflected evaluation on the challenge’s external held-out test cohort. During development, we also generated a labeled prediction file on the training data as a sanity check. These training-set metrics were:

The training-set results should be interpreted only as a sanity check, because the model was evaluated on data it had seen during training. The external leaderboard log loss is the more meaningful indicator of challenge performance.

10 Interpretability and Feature Importance

One advantage of the final Lasso Logistic Regression model is that its learned coefficients provide a direct way to inspect which features contributed most strongly to the linear prediction function. Because the input protein features were anonymized, these coefficients should not be interpreted as validated biological discoveries. However, they are still useful for understanding the model’s internal behavior.

Table 7 lists the top anonymized features by absolute coefficient magnitude from the saved Lasso feature-importance artifact.

Positive coefficients increase the model’s predicted log-odds of ATI, while negative coefficients decrease them, conditional on the standardized feature representation. Since the feature identities are anonymized, we avoided making claims about specific proteins, biomarkers, or pathways.

11 Limitations

Several limitations are important when interpreting this solution:

- The dataset was small relative to the number of input features, which increases overfitting risk.
- Leaderboard feedback guided model iteration, but the external test labels were not available for deeper analysis.
- The protein features were anonymized, limiting biological interpretation of selected coefficients.
- We did not apply additional domain-specific preprocessing or feature engineering beyond the provided processed data and model-specific scaling.
- The final model is linear, so it may not capture nonlinear feature interactions that could be relevant in the underlying biology.
- The training-set metrics are optimistic and should not be treated as evidence of clinical generalization.

12 Conclusion

Team Rockets’ final BKBC hackathon submission used a sparse Lasso Logistic Regression model to predict acute tubular injury from high-dimensional plasma proteomics and clinical covariates. Although the model was relatively simple, it performed well under the challenge’s external leaderboard evaluation, achieving a log loss of **0.322528**. The result supports a practical lesson from the hackathon: in small-sample, high-dimensional biomedical prediction problems, regularized linear models can be highly competitive, especially when the evaluation rewards probabilistic prediction and generalization.

A Repository and Reproducibility Notes

The repository contains all code needed to run the submitted model. No notebook is required for inference. The main files are:

- `README.md`: submission README with setup and command instructions;
- `old.md`: preserved original starter-code documentation;
- `requirements.txt`: Python package requirements;
- `predict.sh`: main prediction entry point;
- `predict.py`: active inference script for the submitted Lasso model;
- `preprocess.py`: data loading and feature/label construction helpers;
- `model.py`: model definitions and constants;
- `evaluate.py`: stratified k-fold cross-validation script;
- `train2.py`: extended training script used for the final Lasso model;
- `weights/`: saved models, feature lists, metadata, and feature-importance files.

The hackathon environment setup described in the repository was:

```
module load medaihack/spring-2026
module load python3/3.12.4
virtualenv .venv
source .venv/bin/activate
pip install -r requirements.txt
```


Model	Rationale for inclusion	Primary concern
XGBoost	Tree ensemble; captures nonlinear interactions; <code>colsample_bytree=0</code> provides aggressive feature subsampling suitable for $p \gg n$	Probability outputs tend to be overconfident without calibration; high risk of overfitting with 6,592 features and only 426 patients
Lasso Logistic Regression	L1 penalty shrinks many protein coefficients to exactly zero, producing sparse solutions; interpretable; established performance in omics settings	May not converge within default iteration limits; risk of underfitting if signal is distributed across many correlated proteins
Elastic Net Logistic Regression	Combines L1 sparsity with L2 stability; handles correlated features better than pure L1; often the strongest single model in high-dimensional biomedical prediction	Requires careful tuning of both C and ℓ_1 -ratio; slower convergence than Lasso
PCA + Logistic Regression	Compresses 6,592 proteins into 50 principal components before classification; reduces noise and correlation; more stable feature representation	Loses interpretability; may discard sparse biological signal if it lies outside the top principal components

Table 3: Candidate model families evaluated in cross-validation.

File	Role
<code>preprocess.py</code>	Loads the CSV and constructs feature matrices and labels
<code>model.py</code>	Defines shared constants and candidate model specifications
<code>evaluate.py</code>	Runs stratified k-fold cross-validation for model comparison
<code>train2.py</code>	Trains selected final models and saves checkpoints/artifacts

Table 4: Development workflow files.

Column	Description
<code>sample_id</code>	Patient/sample identifier
<code>prob_ati</code>	Predicted probability that ATI is present
<code>pred_label</code>	Hard class prediction using a 0.5 threshold
<code>true_label</code>	Included only when the input CSV contains <code>ati</code>

Table 5: Prediction output format.

Metric	Training-set sanity-check value
Log loss	0.282
ROC AUC	0.992
Accuracy at threshold 0.5	0.960

Table 6: Training-set metrics computed from the example prediction output. These are not held-out generalization estimates.

Feature	Coefficient	Absolute coefficient
<code>feature_1002</code>	0.3073	0.3073
<code>feature_4764</code>	0.2348	0.2348
<code>feature_6333</code>	0.2316	0.2316
<code>feature_0419</code>	0.2143	0.2143
<code>feature_2626</code>	0.1961	0.1961
<code>feature_4858</code>	0.1957	0.1957
<code>feature_0124</code>	-0.1915	0.1915
<code>feature_0659</code>	-0.1893	0.1893
<code>feature_2446</code>	0.1889	0.1889
<code>feature_3136</code>	0.1777	0.1777

Table 7: Top anonymized features by absolute Lasso coefficient magnitude.